

# Day 113: Clustering and K-Means

## 1. Day number

Day 113

## 2. Topic name

**Unsupervised Learning and K-Means Clustering**

## 3. Connection

Until now, you mainly worked with **supervised learning**, where training data contains a known answer or **label**.

For example:

| Age | Income | Bought_Product |
|-----|--------|----------------|
| 25  | 30000  | No             |
| 40  | 70000  | Yes            |

Here, Bought\_Product is a label.

Today, suppose we only have:

| Age | Income |
|-----|--------|
| 25  | 30000  |
| 40  | 70000  |

There is no correct category telling us what kind of customer each person is.

We can use **unsupervised learning** to look for useful patterns or groups automatically.

---

## 4. Important topics

### Unsupervised learning

**Unsupervised learning** works with data that does not contain known target labels.

Instead of asking:

| "What is the correct answer?"

we often ask:

| "Are there natural patterns or groups in this data?"

Clustering is one common unsupervised-learning task.

### Cluster

A **cluster** is a group of data points that are relatively similar to each other.

For example:

Cluster 1 → younger customers with lower spending  
Cluster 2 → middle-income regular customers  
Cluster 3 → high-income high-spending customers

These descriptions are interpretations we might make **after** examining the clusters. K-Means itself does not automatically understand concepts such as "high-value customer."

## Centroid

A **centroid** is the center point representing a cluster.

Imagine several customers plotted on a graph. The centroid is roughly the center of one customer group.

**k**

k means:

**| How many clusters should K-Means create?**

For example:

k = 2

means:

Create 2 groups.

And:

k = 3

means:

Create 3 groups.

## Assignment

K-Means assigns every data point to the centroid that is closest to it.

Conceptually:

Customer → nearest centroid → cluster

## Iteration

K-Means does not usually finish after one assignment.

It repeatedly:

```
assigns points
→ updates centroids
→ assigns points again
→ updates centroids again
```

This repetition is called **iteration**.

---

## 5. Foundational notes

A useful distinction is:

```
Supervised Learning
Data → Features + Known Label

Unsupervised Learning
Data → Features only
```

For example, in classification:

```
Age + Income → Will Buy?
```

we already know the training labels.

In clustering:

```
Age + Income → Discover customer groups
```

there is no predefined correct customer group.

Clustering is useful for things such as:

- customer segmentation,
- grouping similar products,
- grouping similar documents,
- discovering patterns in unlabeled data.

An important point is that a cluster is **not automatically a real-world category**. The algorithm only groups points according to the numerical features you give it.

---

## 6. Customer-grouping example

Imagine a store has these customers:

| Customer | Annual Spending | Visits per Month |
|----------|-----------------|------------------|
| A        | 100             | 2                |
| B        | 120             | 3                |
| C        | 110             | 2                |
| D        | 800             | 12               |
| E        | 850             | 11               |
| F        | 900             | 13               |

Even without labels, you can probably notice two rough groups.

One group:

A, B, C

has relatively low spending and fewer visits.

Another group:

D, E, F

has higher spending and more visits.

K-Means can help discover this type of grouping automatically.

If we choose:

$k = 2$

the model tries to create two clusters.

Conceptually:

Cluster 0

A  
B  
C

Cluster 1

D  
E  
F

The numeric cluster names such as 0 and 1 have no special meaning. They are simply identifiers.

## 7. K-Means step by step

Suppose we tell K-Means:

```
k = 2
```

meaning we want two clusters.

### Step 1: Choose initial centroids

The algorithm starts with two candidate center points.

Conceptually:

```
Centroid A
```

```
Centroid B
```

At first, these centers may not be ideal.

### Step 2: Measure which centroid each point is closest to

For every customer, K-Means checks:

```
Is this customer closer to centroid A  
or centroid B?
```

### Step 3: Assign customers to clusters

Customers near centroid A become one group.

Customers near centroid B become another.

For example:

```
Cluster A:
```

```
Customer 1
```

```
Customer 2
```

```
Customer 3
```

```
Cluster B:
```

```
Customer 4
```

```
Customer 5
```

```
Customer 6
```

### Step 4: Recalculate each centroid

Now the algorithm finds a new center for each cluster based on the points assigned to it.

So the centroids move.

### Step 5: Assign the points again

Because the centroids moved, K-Means checks the customers again.

Some assignments might change.

### Step 6: Repeat

The algorithm continues:

```
assign
↓
recalculate centroid
↓
assign again
↓
recalculate again
```

Eventually, the assignments stop changing significantly.

The clustering is then considered complete.

A compact mental model is:

```
Choose centers
↓
Assign points
↓
Move centers
↓
Assign again
↓
Repeat
```

---

## 8. Easy example

Consider this tiny dataset:

| Customer | Spending | Visits |
|----------|----------|--------|
| A        | 10       | 1      |
| B        | 12       | 2      |
| C        | 11       | 1      |
| D        | 80       | 8      |
| E        | 85       | 9      |
| F        | 82       | 8      |

Suppose:

$k = 2$

Looking at the values, you may expect something approximately like:

Group 1:

A, B, C

Group 2:

D, E, F

because A–C have similar feature values, while D–F have similar feature values.

K-Means attempts to discover that pattern using distances between the numerical points.

---

## 9. Problem statement

You have this customer dataset:

| Customer | Purchases per Month | Monthly Spending |
|----------|---------------------|------------------|
| A        | 1                   | 20               |
| B        | 2                   | 25               |
| C        | 2                   | 30               |
| D        | 8                   | 150              |
| E        | 9                   | 160              |
| F        | 10                  | 155              |

Your task is to:

1. Use Purchases per Month and Monthly Spending as the two features.
2. Choose a small value of  $k$ .
3. Group the customers conceptually or using basic K-Means.
4. Inspect which cluster each customer receives.
5. Think about what pattern the clusters may represent.

Do not create labels manually before running K-Means.

---

## 10. Concepts used

This exercise uses:

- unsupervised learning,
- numerical features,
- clustering,
- similarity based on distance,

- K-Means,
- k,
- centroids,
- cluster assignment,
- iteration.

The input could conceptually look like:

```
X = [
    [1, 20],
    [2, 25],
    [2, 30],
    [8, 150],
    [9, 160],
    [10, 155]
]
```

There is no separate:

```
y
```

because we do not have known target labels.

That is a major difference from many supervised-learning examples.

---

## 11. Thought process

Before using K-Means, think through the problem in this order.

**First:** What objects am I trying to group?

```
Customers
```

**Second:** Which characteristics describe them?

```
Purchases per month
Monthly spending
```

These become the features.

**Third:** Do I already know the correct groups?

```
No
```

That makes clustering potentially appropriate.

**Fourth:** How many groups should I start with?

For this tiny exercise, you might try:

```
k = 2
```

**Fifth:** Train K-Means on the feature matrix.

```
model.fit(X)
```

**Sixth:** Inspect the cluster assignment for each customer.

You may see something like:

```
0
0
0
1
1
1
```

Remember that cluster 0 is not inherently "better," "lower," or "worse" than cluster 1.

---

## 12. Beginner-friendly pseudocode

```
START

create a tiny customer dataset

select:
    purchases
    spending

store these columns as X

choose number of clusters k

create K-Means model using k

fit K-Means using X

get cluster assignments

for each customer:
    display customer and cluster number

inspect the groups

END
```

In scikit-learn, the basic workflow is conceptually:

```
from sklearn.cluster import KMeans

prepare X

create KMeans model

fit model on X

get cluster labels
```

Notice:

```
fit(X)
```

instead of the familiar supervised pattern:

```
fit(X, y)
```

There is no known y label here.

---

## 13. Suggested solving approach: scikit-learn

For the exercise, a simple scikit-learn workflow is enough:

1. Create the small dataset
2. Select the two numerical feature columns
3. Create KMeans with k = 2
4. Fit it using the features
5. Read the cluster labels
6. Attach the labels to the customer rows
7. Compare customers belonging to each cluster

You may encounter code structured approximately like:

```
model = KMeans(
    n_clusters=...,
    random_state=...
)
```

Here:

```
n_clusters
```

represents k.

A `random_state` is often supplied in beginner examples so repeated runs are easier to reproduce.

Later you will learn that feature scale can also matter for distance-based algorithms such as K-Means. For today's lesson, keep the dataset simple enough to focus on the basic clustering idea.

---

## 14. Easy edge cases

### Poor choice of k

Imagine there are roughly two natural groups, but you choose:

$k = 5$

K-Means must still create five clusters.

That could split sensible groups into several smaller groups.

Likewise, if several different groups exist but you choose:

$k = 1$

everything will be placed into one cluster.

So choosing k matters.

For today, you do not need advanced methods for choosing it.

### Very small dataset

Suppose you have only three customers:

Customer A  
Customer B  
Customer C

Trying:

$k = 10$

does not make sense because you are asking for more clusters than available data points.

Even when k is technically possible, extremely small datasets may not contain enough information to form meaningful groups.

---

## 15. Hint only

For the six-customer problem, look at the numbers before writing any code.

Ask yourself:

Which customers have similar purchasing and spending patterns?

You may notice that the first few customers are close to one another, while the later customers form another nearby group.

Start with:

$k = 2$

Then think about how K-Means would position one centroid near each group and repeatedly update those centroids.

**Do not worry about manually calculating exact centroid coordinates yet.** The important idea for Day 113 is:

```
No labels
  ↓
Choose k
  ↓
Find centroids
  ↓
Assign nearby points
  ↓
Move centroids
  ↓
Repeat
  ↓
Clusters
```